

Atividade derivação - 26/08

Guilherme Yukio Yamaguchi - 24062624-2

Luis Felipe Kamphorst - 24072626-2

Código (Contador 0-10):

```
#include <stdio.h>
int main() {
    int i;
    for (i = 0; i <= 10; i++) {
        printf("%d\n", i);
    }
    return 0;
}
```

Fonte de referência: ChatGPT - Notação: BNT

Regras gramaticais:

- `<for_statement> ::= for ( <initialization> ; <condition> ; <increment> ) <statement>`
- `<initialization> ::= <identifier> = <integer_literal>`
- `<condition> ::= <expression> <= <integer_literal>`
- `<increment> ::= <identifier> ++`
- `<statement> ::= <compound_statement>`
- `<compound_statement> ::= { <statement_list> }`
- `<statement_list> ::= <statement> | <expression_statement>`
- `<expression_statement> ::= <function_call> ;`
- `<function_call> ::= printf ( <argument_list> )`
- `<argument_list> ::= <string_literal> , <expression>`
- `<expression> ::= <identifier>`
- `<identifier> ::= i`
- `<integer_literal> ::= 0 | 10`
- `<string_literal> ::= "%d\n"`

Fonte: <https://en.cppreference.com/c/language/statements>

Derivação:

```
⇒ for ( <initialization> ; <condition> ; <increment> ) <statement>
⇒ for ( <identifier> = <integer_literal> ; <condition> ; <increment> )
<statement>
⇒ for ( i = <integer_literal> ; <condition> ; <increment> ) <statement>
⇒ for ( i = 0 ; <condition> ; <increment> ) <statement>
⇒ for ( i = 0 ; <expression> <= <integer_literal> ; <increment> ) <statement>
⇒ for ( i = 0 ; <identifier> <= <integer_literal> ; <increment> ) <statement>
⇒ for ( i = 0 ; i <= <integer_literal> ; <increment> ) <statement>
⇒ for ( i = 0 ; i <= 10 ; <increment> ) <statement>
⇒ for ( i = 0 ; i <= 10 ; <identifier> ++ ) <statement>
⇒ for ( i = 0 ; i <= 10 ; i ++ ) <statement>
⇒ for ( i = 0 ; i <= 10 ; i ++ ) <compound_statement>
⇒ for ( i = 0 ; i <= 10 ; i ++ ) { <statement_list> }
⇒ for ( i = 0 ; i <= 10 ; i ++ ) { <expression_statement> }
⇒ for ( i = 0 ; i <= 10 ; i ++ ) { <function_call> ; }
⇒ for ( i = 0 ; i <= 10 ; i ++ ) { printf ( <argument_list> ) ; }
⇒ for ( i = 0 ; i <= 10 ; i ++ ) { printf ( <string_literal> , <expression> )
; }
⇒ for ( i = 0 ; i <= 10 ; i ++ ) { printf ( "%d\n" , <expression> ) ; }
⇒ for ( i = 0 ; i <= 10 ; i ++ ) { printf ( "%d\n" , <identifier> ) ; }
⇒ for ( i = 0 ; i <= 10 ; i ++ ) { printf ( "%d\n" , i ) ; }
```

Forma concreta:

```
for (i = 0; i <= 10; i++) {
    printf("%d\n", i);
}
```

Explicação:

O comando for da linguagem C é usado para repetir um conjunto de comandos. Neste exemplo, a variável *i* começa com o valor 0. A condição *i* <= 10 indica que a repetição continua enquanto *i* for menor ou igual a 10. Depois de cada repetição, *i*++ aumenta o valor de *i* em 1. Entre { e } está o bloco de comandos que será repetido. Dentro dele, `printf("%d\n", i)` mostra na tela o valor atual de *i*. A derivação mostra que esse código é válido porque pode ser construído passo a passo usando as regras da gramática da linguagem C.